APPENDIX

Table 7: Utility functions and corresponding constraints.*(- : not applicable.)*

Name	Description	Constraints
`checkThrow(<fun>, <flag>)`	Examines whether `<fun>` throws or does not throw an exception, as indicated by `<flag>`.	$TH_{last} = flag$
`checkCall(<fun>, <fun>)`	Examines whether the first input `<fun>` calls the second input `<fun>`.	$CA_{fun2,last} = true$
`checkEmit(<fun>, <event>)`	Examines whether `<fun>` emits `<event>`.	$EM_{event,last} = true$
`checkEndValue(<fun>, <value>, <value>)`	Examines whether the first input `<value>` equals the second input `<value>` at the end of `<fun>`.	$value1_{last} = value2_{last}$
`checkOrder(<op>, <op>, <order>)`	Examines whether the two `<op>`s follow the given `<order>`.	$(O_{op1} < O_{op2} \wedge O_{op3} < O_{op4}) \vee (O_{op1} > O_{op2} \wedge O_{op3} > O_{op4})$
`getFieldValue(<field_name>, (, <key>)*)`	Returns the value of contract field `<field_name>`, if field `<field_name>` is an array or mapping, then `<key>`s are used to return one of its elements.	-
`getFieldName(<anchor_function>)`	Computes the name of a contract field by checking which contract field is accessed inside `<anchor_function>`.	-
`getPara(<fun>|<event>, <index>)`	Returns the parameter of the input `<fun>` or `<event>` at the specified `<index>`.	-
`getOrder(<op>, <op>)`	Computes the order of the two input `<op>`s.	-
`checkChange(<fun>, <field_name>, (, <key>)*)`	Checks whether `<fun>` modifies the value of the input contract field.	$BC_{field_name,last}$
`checkMint(<fun>, <field_name>)`	Checks whether `<fun>` exclusively increases the value of the contract field without decreasing it.	$BI_{field_name,last} \wedge \neg BD_{field_name,last}$
`checkBurn(<fun>, <field_name>)`	Checks whether `<fun>` exclusively decreases the value of the contract field without increasing it.	$\neg BI_{field_name,last} \wedge BD_{field_name,last}$

```
Given an ERC, which contains a list of functions, events, and rules. Each function has its own rules, usually above their Solidity declaration. Rules indicating throw or check usually have patterns like
"""
{sub} [must] [throw] [condition]
[action] [must] [throw]
caller [must] be approved to [action]
[action] are considered invalid
"""
Other patterns that serve similar purposes also count.

Rules:"""
{{description}}
{{API}}
"""
List the conditions that need to be thrown or checked in the given rules for function {{API}} in a JSON array with the following format:

if: <string, condition to throw or check>
throw: <boolean, whether it needs to throw or should not be thrown>

If there is no such rule, simply left empty.
```

Figure 6: The template for prompts extracting TP rules. ***("{{description}}": text descriptions precede a function declaration in an ERC document. "{{API}}": the function declaration itself.)***

```
For {{API}}
rule: {{rule}}
{{args}}
By using the following JSON schema of the configuration for the rule verification:
{{JSON schema for EBNF}}
{{anchor function lists}}
Generate the JSON for the rule. {"" if <args rule> else "Do not generate the arg part if there is no arg in the rule."}
```

Figure 7: The prompt template to translate a rule in natural language into the EBNF grammar. ***("{{API}}": the declaration in source code of a function or event. "rule": the extracted natural language rule for that function or event. "args" describes the possible arguments for the rule in natural language.)***